COMPUTER ENGINEERING

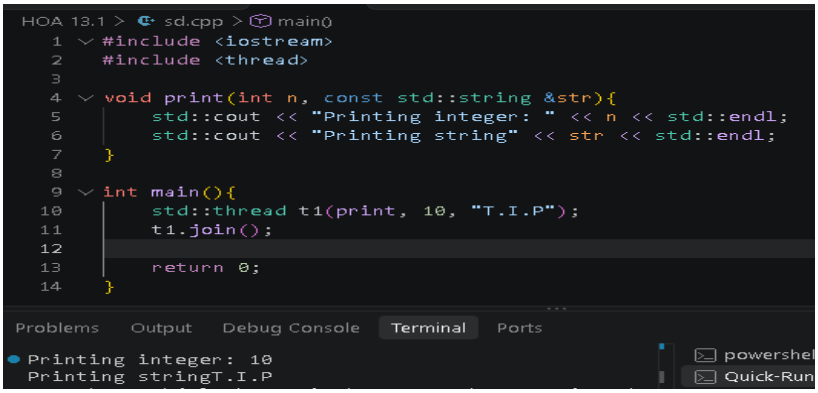
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	20/10/2026	**DATE SUBMITTED:**	20/10/2026
NAME:	Quiming, Kadaphy James A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

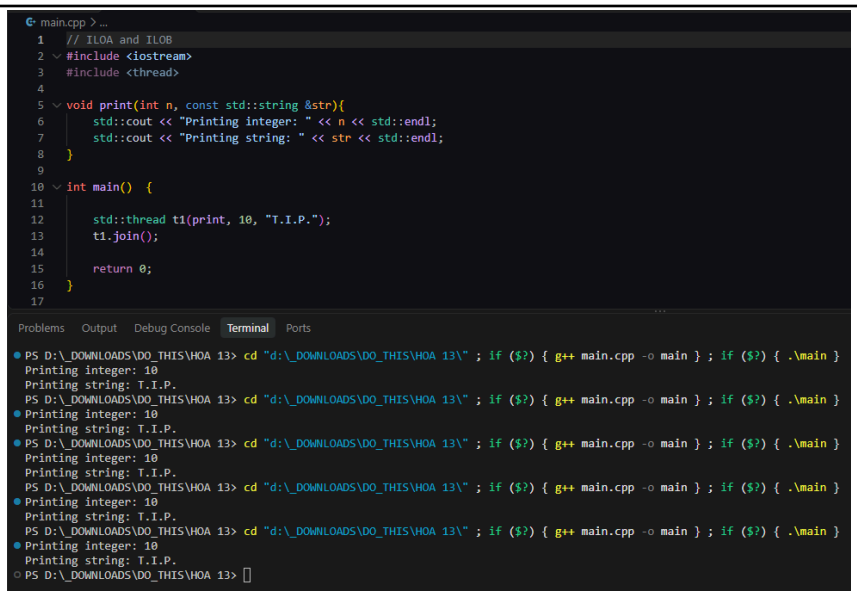
ACTIVITY NO. 13 PARALLEL ALGORITHMS AND MULTITHREADING

1. PROCEDURE OUTPUT

Larracas Screenshot



Quiming Screenshot



Analysis	Larracas	Same in my device, both integer and string appear at the same time.
	Quiming	Both integer(10) and string("T.I.P.") both appear in the output at the same time in order consistently.

Table 13-1. Simple One-Threaded Example

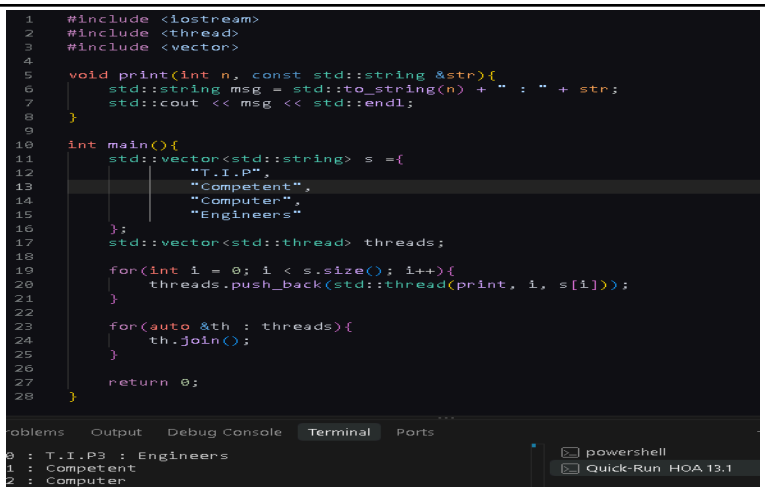
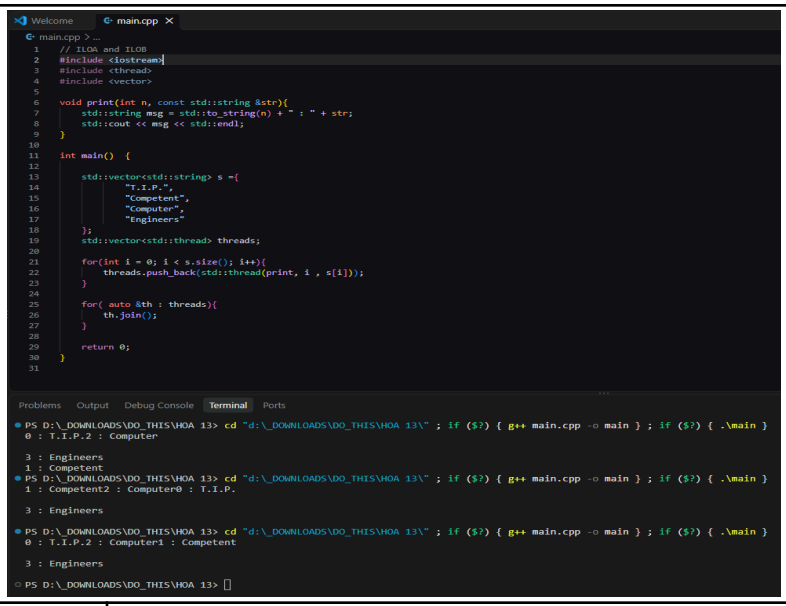
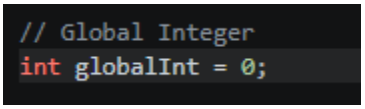
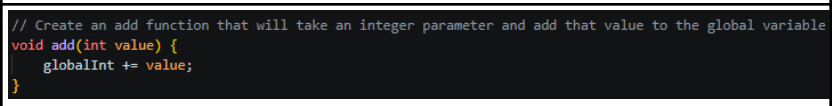
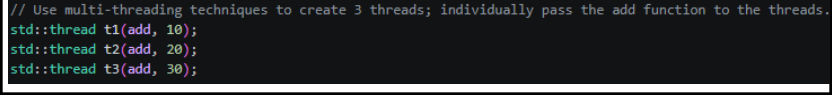
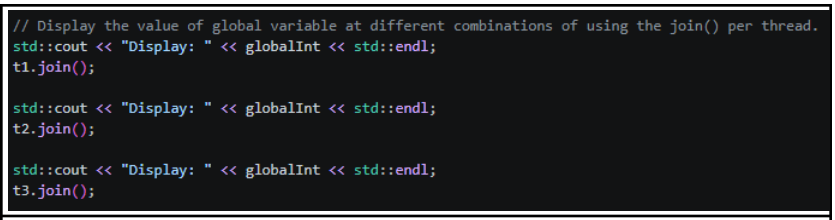
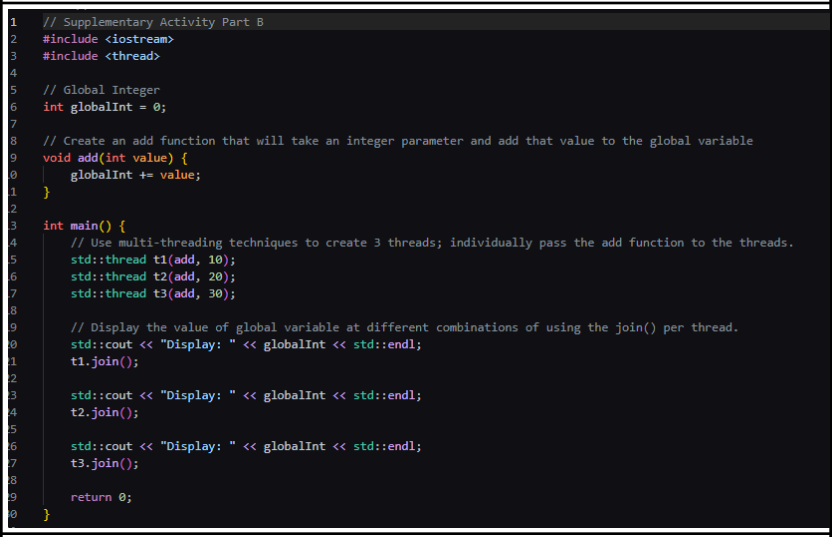
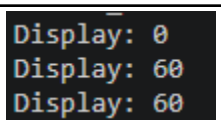
Larracas Screenshot		
Quiming Screenshot		
Analysis	Larracas	My output shows the list of strings that doesn't match the in order of words because the system run based on my device speed. Since I have a lot of applications in the background, the output seems to be slower because the output should've shown the words sequentially in one line.
	Quiming	The lists of strings randomly appear in the output.

Table 13-2. Multithreaded Example

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A	
1. Write a definition of multithreading and its advantages/disadvantages.	
Quiming	Multithreading from its name multi for multiple and threading, allows a program to run multiple threads in one process. Although multithreading has faster execution, better resource usage and faster responsiveness there are issues that can occur such as data races where the program tries to overlap the data resulting in the output looking randomized.
Larracas	Multithreading allows single process to execute multiple threads to perform different tasks. The advantages are faster execution, better resource utilization and improved responsiveness. On the other hand, the disadvantages of itm is it has deadlocks and difficulties in magaing shared data which can lead to inaccurate results depending on how fast is your device.
2. Rationalize the use of multithreading by providing at least 3 use-cases.	
Quiming	1. Web servers: Allows the handling of multiple users at once. 2. GUI Applications: Allows the GUI to do multiple tasks like keeping the GUI responsive while also doing other task like loading files. 3. Image Processing: Process different parts of an image simultaneously to load the image faster.
Larracas	● Online gaming allows games to handle tasks such as sound, graphics, and user input. ● Video editing allows different video effects to be processed simultaneously, making video render faster. ● Database systems allows to handle multiple queries from the different users, improving efficiency and response time.
3. Differentiate between parallelism and concurrency.	
Quiming	Concurrency means handling multiple tasks at the same time even if they take turns running, Parallelism means running multiple tasks at the same time using different CPU cores.
Larracas	Parallelism has the ability to execute multiple tasks at exactly the same time using multiple CPU cores or processing units. On the other hand, concurrency has the ability of a program to manage multiple tasks during overlapping periods.

Part B	
Create a global variable of type integer	
Create an add function that will take an integer parameter and add that value to the global variable	
Use multi-threading techniques to create 3 threads; individually pass the add function to the threads.	

Display the value of global variable at different combinations of using the join() per thread.	
Output	 

```
// Display the value of global variable at different combinations of using the join() per thread.
std::cout << "Display: " << globalInt << std::endl;
t1.join();

std::cout << "Display: " << globalInt << std::endl;
t2.join();

std::cout << "Display: " << globalInt << std::endl;
t3.join();
```

```
// Supplementary Activity Part B
#include <iostream>
#include <thread>

// Global Integer
int globalInt = 0;

// Create an add function that will take an integer parameter and add that value to the global variable
void add(int value) {
    globalInt += value;
}

int main() {
    // Use multi-threading techniques to create 3 threads; individually pass the add function to the threads.
    std::thread t1(add, 10);
    std::thread t2(add, 20);
    std::thread t3(add, 30);

    // Display the value of global variable at different combinations of using the join() per thread.
    std::cout << "Display: " << globalInt << std::endl;
    t1.join();

    std::cout << "Display: " << globalInt << std::endl;
    t2.join();

    std::cout << "Display: " << globalInt << std::endl;
    t3.join();

    return 0;
}
```

```
Display: 0
Display: 60
Display: 60
```

Analysis

Quiming	The reason why the first display is 0 is that the `int main()` function creates the threads and print those threads right away. Starting a thread takes a little more time so none of the threads that we have doesn't run the `void add(int value)` yet when the main function reads the variable. The second Display which outputs 60 is that the `t1.join();` makes the main function wait for T1 while T2 and T3 finish, all the 3 threads add their values resulting in 10 + 20 + 30 = 60. Lastly, T2 and T3 are already finished executing so `t2.join();` returns instantly.
Larracas	The first output is 0 because the three threads are created, but they may not have started executing the add function when the main thread prints the value of global integer creating a thread does not guarantee that it will execute immediately. The second output is 60 because t1.join function makes the main thread wait until thread 1 finishes. The third output remains 60 because threads 2 and 3 have already finished executing by the time their respective join function calls are reached. Thus, making the value of global integer does not change.

Part C

4. Write a definition of multithreading and its advantages/disadvantages.

SA_PartC.cpp > ...

```cpp
// Supplementary Activity Part C (Bottom-Up DP)
#include <iostream>
#include <thread>
#include <vector>

// Bottom-Up DP
int getMinSteps(int n){
    int dp[n + 1], i;
    dp[1] = 0;  // trivial case
    for(i = 2; i <= n; i++){
        dp[i] = 1 + dp[i - 1];
        if(i % 2 == 0) dp[i] = std::min(dp[i], 1 + dp[i / 2]);
        if(i % 3 == 0) dp[i] = std::min(dp[i], 1 + dp[i / 3]);
    }
    return dp[n];
}

// Saves the answer in its own variable (just like the add function in Part B)
void run(int n, int &result){
    result = getMinSteps(n);
}

int main(){
    int r1 = 0, r2 = 0, r3 = 0;

    std::thread t1(run, 10, std::ref(r1));
    std::thread t2(run, 100, std::ref(r2));
    std::thread t3(run, 1000, std::ref(r3));

    t1.join();
    t2.join();
    t3.join();

    std::cout << "Min steps for n = 10: " << r1 << std::endl;
    std::cout << "Min steps for n = 100: " << r2 << std::endl;
    std::cout << "Min steps for n = 1000: " << r3 << std::endl;

    return 0;
}
```

Problems Output Debug Console **Terminal** Ports

```
PS D:\_DOWNLOADS\DO_THIS\HOA 13> cd "d:\_DOWNLOADS\DO_THIS\HO
Min steps for n = 10: 3
Min steps for n = 100: 7
Min steps for n = 1000: 9
PS D:\_DOWNLOADS\DO_THIS\HOA 13>
```

Analysis (Quiming)	For n = 10, it takes 3 steps: 10 > 9 > 3 > 1. The dp loop must run in order because each value depends on the previous ones., The 3 threads solve different problems without interfering with each other. The results are printed after all threads finish using join().

Analysis (Larracas)	The program calculates the minimum number of steps needed to reach the final value using dynamic programming for n = 10. The result is 3 steps, following the sequence 10 to 9 to 3 to 1. The program then creates three threads to calculate the minimum steps for 10, 100 and 1000. Each thread stores its result in a separate variable, allowing the calculations to run continuously without interfering with one another. The join function prints all of the results after all threads are finish.
Conclusion (Larracas)	In conclusion, I still have more room to practice with multithreading since I am still absorbing the concepts of it but I still learn something from it. Thus, the importance of this lesson is that multithreading depends on the device that you use and how fast does your device is because multithreading works by running different tasks at the same time depending on how fast your device or how much does your device can handle the memory of it.
Conclusion (Quiming)	I learned how multithreading allows multiple tasks to run simultaneously. I also learned some functionalities and uses of join() like it makes the main thread wait doe the other threads to finish.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **n/a**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **n/a**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **n/a**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.